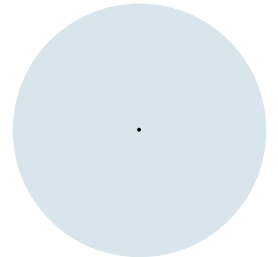


# **Variablen, Literale und Zuweisungen**



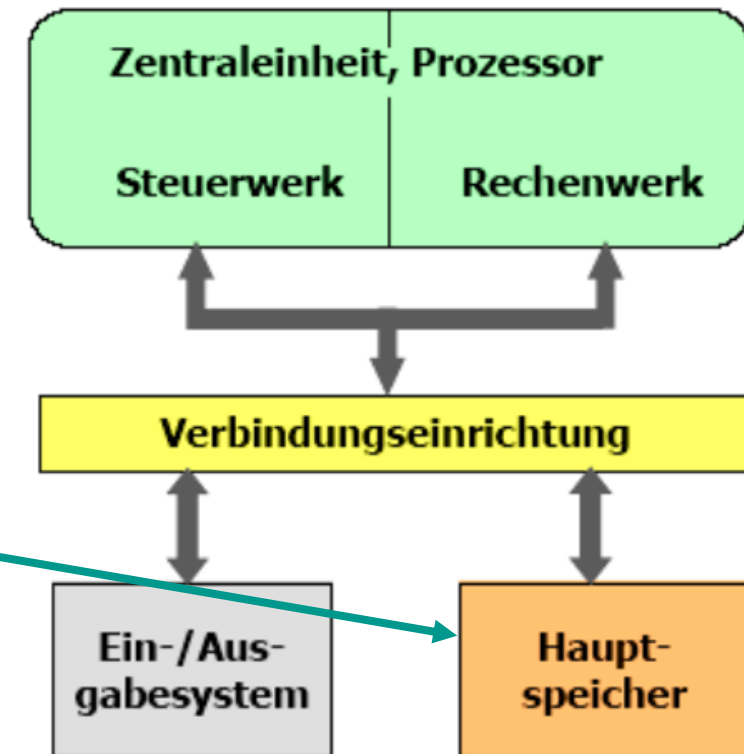
# Variablen

- **Benannte** und **typisierte** Speicherbereiche, im Hauptspeicher, die Werte des angegebenen Typs enthalten können.
- **Variablen** müssen vor ihrer Benutzung deklariert werden:

```
int age;  
double averageGrade;
```

(erst **Typ**, dann **Name**)

- Variablen müssen initialisiert werden, z.B. durch Zuweisung eines initialen Werts als Literal (mehr dazu gleich)



# Ganzzahltypen

| Typ   | Länge  | Kleinste darstellbare Zahl | Größte darstellbare Zahl  |
|-------|--------|----------------------------|---------------------------|
| byte  | 8 Bit  | -128                       | 127                       |
| short | 16 Bit | -32.768                    | 32.767                    |
| int   | 32 Bit | -2.147.483.648             | 2.147.483.647             |
| long  | 64 Bit | -9.223.372.036.854.775.808 | 9.223.372.036.854.775.807 |

```
int germanPopulation = 83846319;
```

```
byte age = 24;
```

```
jshell>
```

# Gleitkommazahltypen

- Größerer Wertebereich als Integer, aber weniger genau

| Typ    | Länge  | Kleinste darstellbare Zahl | Größte darstellbare Zahl |
|--------|--------|----------------------------|--------------------------|
| float  | 32 Bit | ca. $(-2) \times 2^{127}$  | ca. $2 \times 2^{127}$   |
| double | 64 Bit | ca. $-10^{308}$            | ca. $10^{308}$           |

```
double valueAddedTax = 1.18;
```

```
jshell>
```

# Namenskonvention

- Sprechende Namen, z.B. isEmpty statt b17
  - Lange Namen erzeugen keinen langen Maschinencode
- "Camel Case"
  - Variablennamen beginnen mit einem Kleinbuchstaben
  - Zusammengesetzte Namen => Anfangsbuchstaben der Folgewörter groß

```
int germanPopulation;  
long volumeOfLakeConstanceInLiter;
```



# Literale

- Typisierte Konstanten, die u.a. verwendet werden können, um Variablen zu initialisieren.

| Typ    | Beispielliterale | evtl. Suffix        |
|--------|------------------|---------------------|
| byte   | 127, -128        |                     |
| short  | 32456            |                     |
| int    | 60273            |                     |
| long   | 13292367454L     | L, l                |
| float  | 87.363f          | F, f                |
| double | 37.266, 37.266D  | (default), D, d, e3 |

← klein L besser nicht verwenden,  
Verwechslungsgefahr mit 1

# Literale, Besonderheiten

- Das Zeichen \_ kann man als (Tausender-)Trennzeichen für lange Zahlen benutzen:

```
long germanPopulation = 83_846_319L;
```

- Durch Voranstellung von 0x, 0b oder einfach nur 0 kann man Literale in anderen Zahlssystemen angeben:

```
int nr = 0xCAFEBAFE;           // Hexadezimalzahl  
int someNumber = 0b010010010;  // Binärzahl  
int someNumber = 011;          // Oktalzahl
```

- Vorsicht:  $11_8 = 1 \cdot 8^1 + 1 \cdot 8^0 = 8 + 1 = 9_{10}$ , also entspricht 011 der Zahl 9 in Dezimaldarstellung, nicht der Zahl 11 (Fehlerquelle!)

Fun fact zu CAFEBABE: <https://dzone.com/articles/the-magic-word-in-java-cafebabe>

# Zuweisungen

- Eine Zuweisung erlaubt, einer **Variablen**  $v$  einen **Wert**  $w$  zuzuweisen.
- $w$  muss **denselben** Typ haben wie  $v$  (oder einen **engeren**)
- Java:  $v = w$ ;
- $=$  in Java ist Zuweisung, nicht Prüfung auf Gleichheit (das ist  $==$ )



# Beispiel Deklaration und Initialisierung mit Literal

```
int intNumber;  
long longNumber;  
double averageGrade;
```

Deklaration

```
intNumber = 42;  
longNumber = 2015L;  
averageGrade = 2.5;
```

```
int intNumber = 42;  
double averageGrade = 2.5;
```

in einem Schritt

jshell>

# Beispiel Zuweisung Wert aus anderer Variable

- Zuweisung des Wertes einer anderen Variable:

```
int lucysFavoriteNumber;  
lucysFavoriteNumber = 42;  
int petersFavoriteNumber;  
petersFavoriteNumber = lucysFavoriteNumber;
```

- in einem Schritt:

```
int lucysFavoriteNumber = 42;  
int petersFavoriteNumber = lucysFavoriteNumber;
```

jshell>

# Beispiel inkompatible Zuweisung

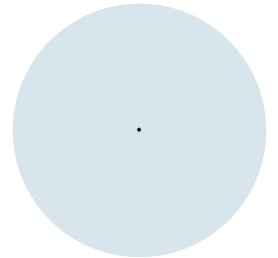
```
int age;  
age = 22.5;
```

Typ int  $\neq$  Typ double  $\rightarrow$  Compilerfehler!

Hinweis: Manche Zuweisungen unterschiedlicher Typen sind aber möglich, z.B. einen int-Wert an eine long-Variable. Mehr dazu später!

jshell>

# Operatoren



# Operatoren, Stelligkeit

- Ein **Operator** ist eine in eine Programmiersprache **eingebaute Funktion**, die auf einem oder mehreren **Operanden** arbeitet.
- Operatoren dienen dazu, aus Werten neue **komplexere Werte** zu **berechnen**.
- Die Anwendung eines Operators auf einen oder mehrere Operanden heißt Operation.
- Die **Stelligkeit** eines Operators gibt die **Anzahl der Operanden** an.
- Java kennt
  - Einstellige/**unäre** Operatoren
  - Zweistellige/**binäre** Operatoren
  - Dreistellige/**ternäre** Operatoren

# Beispiele

- Diese Operatoren kennen wir schon:

Zuweisungsoperator

```
int zahl = 42;
```

- In Java lässt sich mit den Operatoren +, -, \*, / und Klammern rechnen:

```
double radius = 5.0;  
double umfang = 2 * Math.PI * radius;  
double durchschnitt = (2.0 + 1.0 + 1.3) / 3.0;
```

# Grundlegende Eigenschaften von Operatoren in Java

- **Position:**

- Bei **Präfix-Darstellung** steht der Operator **vor** den Operanden.

- $-x$

- Bei **Infix-Darstellung** steht der Operator **zwischen** den Operanden.

- $a + b$

- $x = 5$

- Bei **Postfix-Darstellung** steht der Operator **hinter** den Operanden.

- $x++$

- **Typisierung:**

- Operatoren sind typisiert, d.h. erwarten **Operanden** bestimmten Typs und liefern ein **Ergebnis** bestimmten **Typs**.

- Z.B.

- $+$  erwartet numerische Operanden (oder Strings)
    - $\&\&$  erwartet boolesche Operanden

# Operatorengruppen

- In Java gibt es
  - Zuweisungsoperatoren
  - arithmetische Operatoren
  - Bitoperatoren (lassen wir heute aus)
  - logische Operatoren (kommen später)
  - Vergleichsoperatoren
  - Inkrement-/Dekrement-Operatoren
  - Bedingungsoperator
  - Objekt- und Klassenoperatoren (machen wir später)



# Zuweisungsoperator

- den Zuweisungsoperator = kennen wir schon
  - = arbeitet auf numerischen Werten, boolean, char und auf Objektreferenzen
  - binärer Operator (zwei Stellen: linke Seite und rechte Seite)
  - Beispiele:
    - Variable number bekommt den Wert 5 zugewiesen  
`number = 5;`
    - Variable anotherNumber bekommt den Wert der Variable number zugewiesen  
`anotherNumber = number;`

jshell>

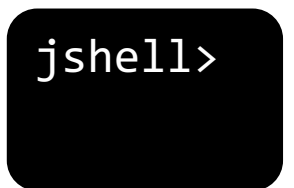
# Arithmetische Operatoren

| Operator | Beispiel | Bedeutung                                                                                                                           |
|----------|----------|-------------------------------------------------------------------------------------------------------------------------------------|
| +        | $x + y$  | Addition                                                                                                                            |
| -        | $x - y$  | Subtraktion                                                                                                                         |
| *        | $x * y$  | Multiplikation                                                                                                                      |
| /        | $x / y$  | Ganzzahl- oder Gleitkomma-Division<br>Ganzzahl: $7 / 4 = 1$<br>Gleitkomma: $7.0 / 4.0 = 1.75$<br>Angewendete Arithmetik typabhängig |
| %        | $x \% y$ | Modulo, Rest der Ganzzahldivision $x / y$<br>$7 \% 4 = 3$                                                                           |
| -        | $-x$     | Negation, einstelliges Minus                                                                                                        |

Punkt vor Strich – ansonsten Klammerung möglich (mehr dazu später):

`1 + 2 * 3 // ergibt 7`

`(1 + 2) * 3 // ergibt 9`

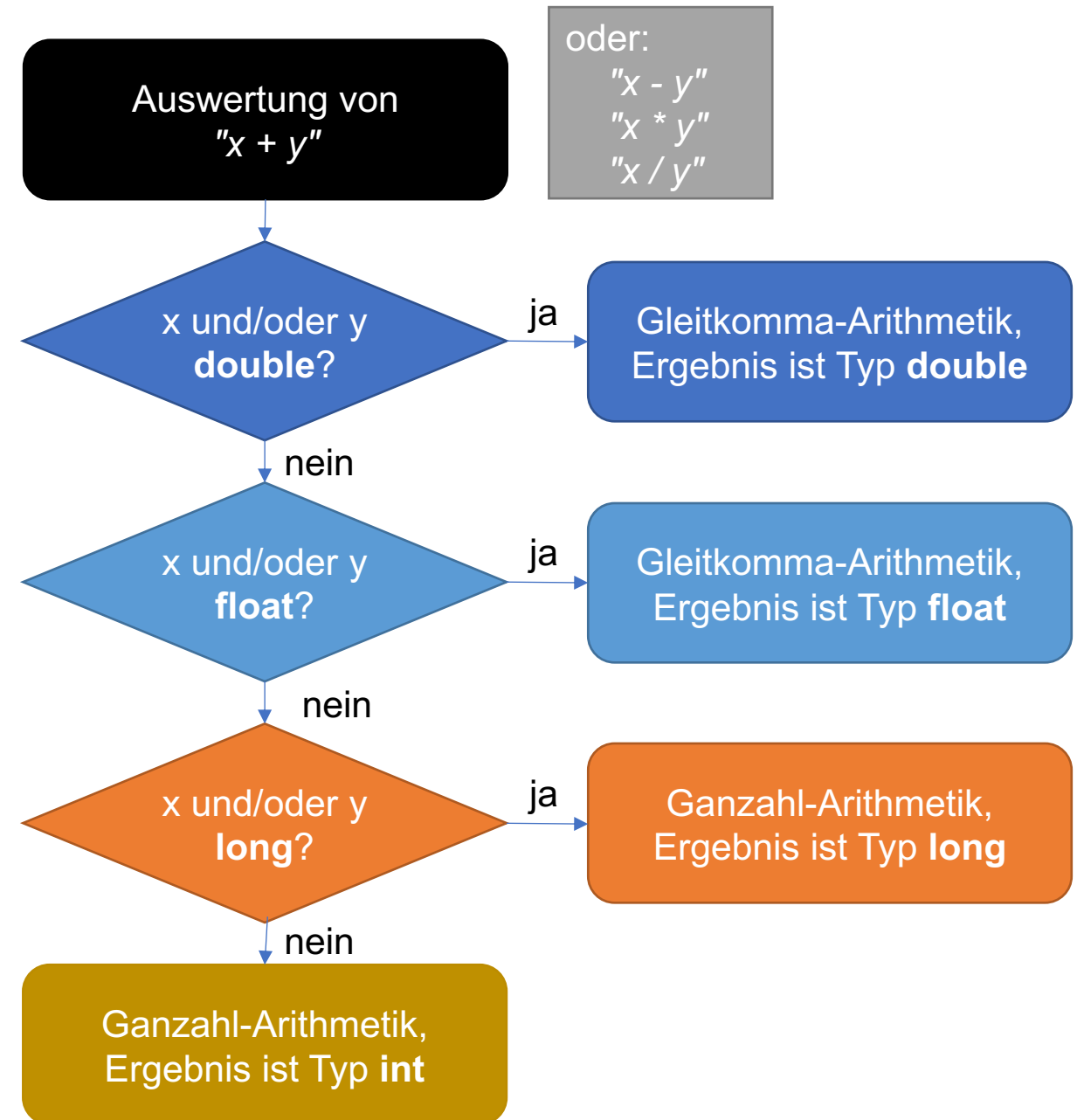


code01.

Arithmetic  
Operators

# Arithmetische Operatoren

- Arbeiten auf numerischen Operanden, also byte, short, int, long, float, double
- Oder auf char (interpretiert als vorzeichenlose 2-Byte-Ganzzahl)
- Ergebnistyp abhängig von Datentypen der Operanden, siehe rechts
- Beispiele:
  - $7.0 + 3.0 \rightarrow 10.0$  (double)
  - $12.0 - 10 \rightarrow 2.0$  (double)
  - $17.0f * 10L \rightarrow 170.0f$  (float)
  - $3L + 10 \rightarrow 13L$  (long)
  - $29 - 5 \rightarrow 24$  (int)



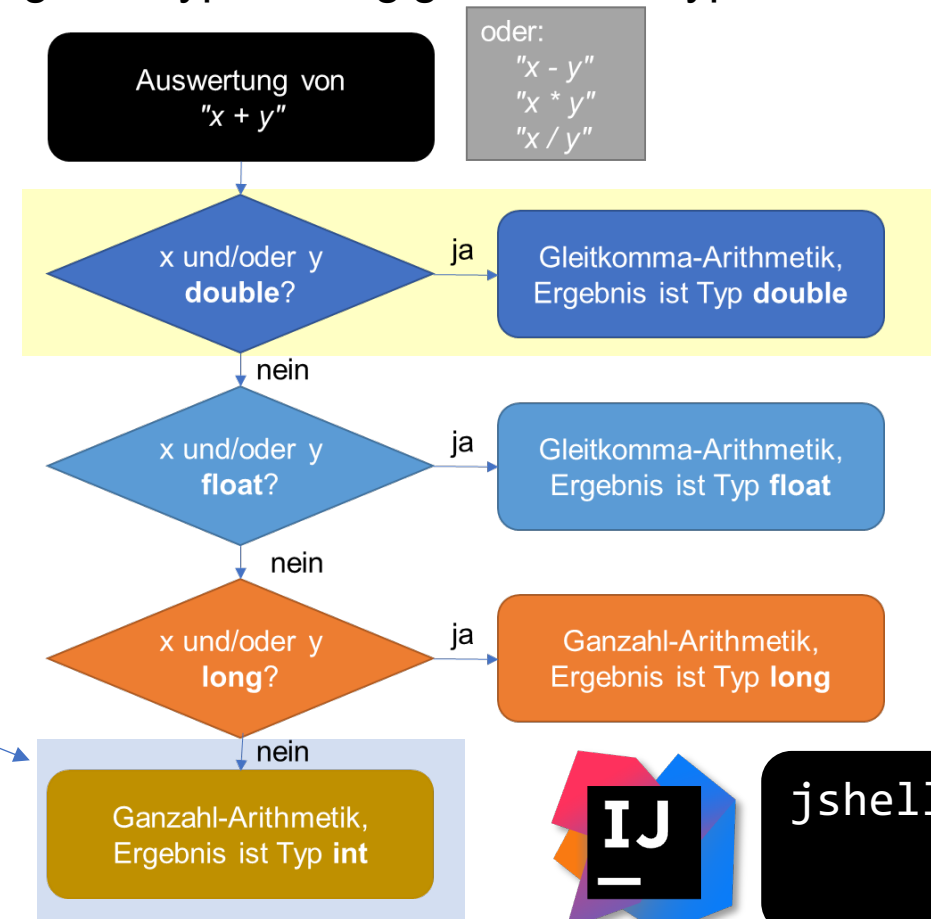
# Häufiger Fehler: Unerwartete Ganzzahldivision

```
int wert = 45;  
double halberWert = wert / 2;  
System.out.println(halberWert);
```

- erwartete Ausgabe: 22.5
- tatsächliche Ausgabe: 22
- Warum?
  - Beide Operanden sind Typ int
- Was tun?
- Gleitkomma-Arithmetik erzwingen!

```
int wert = 45;  
double halberWert = wert / 2.0;  
System.out.println(halberWert);
```

**Erinnerung:** Ergebnistyp abhängig von Datentypen der Operanden:



code01.

Int

Division1

jshe11>

# Häufiger Fehler: Unerwartete Ganzzahldivision

- Weitere Möglichkeit, Gleitkomma-Division zu erzwingen:
  - einen der Parameter auf double casten

```
double result2 = (double) nrOfCookies / nrOfPersons;
```

- Alle Zeilen *bis auf die erste Berechnung (!)* rechnen mit Gleitkomma-Division

```
// ergibt 4
System.out.println("9 / 2 = " + (9 / 2));

// ergibt 4.5
System.out.println("9.0 / 2 = " + (9.0 / 2));
System.out.println("9 / 2.0 = " + (9 / 2.0));
System.out.println("9.0 / 2.0 = " + (9.0 / 2.0));
System.out.println("(double) 9 / 2 = " + ((double) 9 / 2));
```



code01.  
Int  
Division2

jshell>

# Zuweisung und Berechnung kombiniert

- Zusammengesetzter Zuweisungsoperator kombiniert Zuweisung mit Berechnung,

- Beispiel:

`number += 5;` ist kurz für `number = number + 5;`

jshe11>

- Allgemein:  $x \text{ <op> } y$  entspricht  $x = x \text{ <op> } y$ 
  - <op> steht für +, −, \*, /, %
  - Zusammengesetzte Zuweisungsoperatoren arbeiten auf numerischen Werten und char
  - += funktioniert auch bei Strings
  - Es handelt sich um sogenannten "Syntaktischen Zucker" (*syntactic sugar*)

**Syntaktischer Zucker** sind Syntaxerweiterungen in Programmiersprachen, welche der Vereinfachung von Schreibweisen dienen. Diese Erweiterungen sind alternative Schreibweisen, die aber nicht die Ausdrucksstärke und Funktionalität der Programmiersprache erweitern. (wikipedia)

# Zuweisungsoperatoren, Übersicht

| Operator | Beispiel   | Bedeutung                               |
|----------|------------|-----------------------------------------|
| =        | $x = y$    | Weist der Variablen x den Wert von y zu |
| +=       | $x += y$   | $x = x + y$                             |
| -=       | $x -= y$   | $x = x - y$                             |
| *=       | $x *= y$   | $x = x * y$                             |
| /=       | $x /= y$   | $x = x / y$                             |
| %=       | $x \% = y$ | $x = x \% y$                            |



code01.  
Combined  
Assignment  
Operators

# Auswertungsreihenfolge

- Ein Ausdruck kann mehrere Operatoren hintereinander enthalten.
- In welcher Reihenfolge wird ausgewertet?

```
int i = 5;  
int j = 8 + 2 * ++i - 1;
```

```
jshell>
```



# Auswertungsreihenfolge

- Prioritäten für Operatoren, z.B. "Punkt vor Strich"
- Auswertungsreihenfolge für Operatoren gleicher Priorität ("Assoziativität")
  - rechts-assoziativ, Auswertung von rechts
  - links-assoziativ, Auswertung von links
- Per Klammerung () gewünschte Auswertungsreihenfolge immer bestimmbar
- Komponenten-zugriffe und Methodenaufrufe haben sogar noch höhere Priorität

Operator Precedence

| Operators            | Precedence                                                    |
|----------------------|---------------------------------------------------------------|
| postfix              | <i>expr</i> ++ <i>expr</i> --                                 |
| unary                | ++ <i>expr</i> -- <i>expr</i> + <i>expr</i> - <i>expr</i> ~ ! |
| multiplicative       | * / %                                                         |
| additive             | + -                                                           |
| shift                | << >> >>>                                                     |
| relational           | < > <= >= instanceof                                          |
| equality             | == !=                                                         |
| bitwise AND          | &                                                             |
| bitwise exclusive OR | ^                                                             |
| bitwise inclusive OR |                                                               |
| logical AND          | &&                                                            |
| logical OR           |                                                               |
| ternary              | ? :                                                           |
| assignment           | = += -= *= /= %= &= ^=  = <<= >>= >>>=                        |

# Assoziativität

- Auswertungsreihenfolge für Operatoren gleicher Priorität
- Alle Zuweisungsoperatoren werden von rechts nach links ausgewertet ("rechts-assoziativ")  
 $a = b = c \rightarrow b = c, \text{ dann } a = b$
- Alle anderen Operatoren werden von links nach rechts ausgewertet ("links-assoziativ")  
 $a + b - c \rightarrow (a + b) - c$

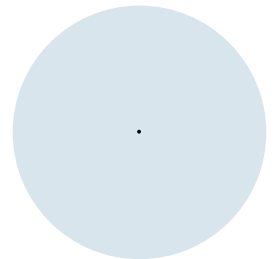


code01.

Left

Association

# Zahldarstellung



# Ausgangsfrage

- Wie stellt man (große) Zahlen **einfach**, **platzsparend** und **rechnergeeignet** dar?



# Abzählen

- I, II, III, IIII, IIIII, IIIIII, IIIIII, IIIIIII, IIIIIIII
- Auf Anhieb kann ein Mensch höchstens kleine Zahlen erkennen
- **Zeit- und platzaufwendig** für große Zahlen

# Strichlisten

- II, IIIII, IIIII I, IIIII IIIII II
- Formatierung in Gruppen, getrennt durch Leerzeichen
- Leichter zu erkennen
- **Zeit- und platzaufwendig** für große Zahlen

# Römische Ziffern

- Symbole:
  - 1: I      5: V      10: X      50: L
  - 100: C      500: D      1.000: M
- Wertigkeit ergibt sich durch Reihenfolge
  - Kleine Zahl rechts von großer Zahl → plus
  - Kleine Zahl links von großer Zahl → minus
  - Höchstens drei gleiche Symbole nebeneinander
- Viel kompakter als Strichlisten
- **Kompliziert** (insbesondere das Rechnen damit)
- Beispiele
  - VI
  - IX
  - XLII
  - MCMLXXXIV
  - MMXIV
  - MMMCMXCIX

# Dezimalsystem

- Das Dezimalsystem ist ein Stellenwertsystem.
- Wert einer Ziffer hängt davon ab, an welcher Stelle sie in der Zahl steht.
- Beispiel:

$$\begin{aligned} 5.582 &= 5 \times 1.000 + & 5 \times 100 + & 8 \times 10 + & 2 \times 1 \\ &= 5 \times 10^3 + & 5 \times 10^2 + & 8 \times 10^1 + & 2 \times 10^0 \end{aligned}$$

- Ziffern: 0, 1, 2, 3, 4, 5, 6, 7, 8, 9
- Basis: 10
- Stellenwerte: 1, 10, 100, 1.000  
( $10^k$  für k-te Stelle von rechts)
- **Kompakte, einfache Darstellung auch großer Zahlen**
- **Einfaches Rechnen** möglich (schriftliche Addition, ...)



# Stellenwertsysteme

- Stellenwertsysteme mit beliebiger Basis
- $z_3 z_2 z_1 z_0 = z_3 \cdot b^3 + z_2 \cdot b^2 + z_1 \cdot b^1 + z_0 \cdot b^0$
- Basis:  $b$
- Ziffern:  $0, 1, 2, \dots, b-1$
- Stellenwerte:  $b^0=1, b^1, b^2, b^3, \dots$
- Übliche Stellenwertsysteme in der Programmierung:

Beispiel Dezimalsystem:

- $5\ 5\ 8\ 2 = 5 \times 10^3 + 5 \times 10^2 + 8 \times 10^1 + 2 \times 10^0$
- $b = 10$
- Ziffern  $0, 1, \dots, 10-1 = 9$
- Stellenwerte  $1, 10, 100, 1000, \dots$

| System      | Basis | Ziffern  | Stellenwerte           |
|-------------|-------|----------|------------------------|
| Binär/Dual  | 2     | 0, 1     | 1, 2, 4, 8, 16, ...    |
| Oktal       | 8     | 0-7      | 1, 8, 64, 512, ...     |
| Dezimal     | 10    | 0-9      | 1, 10, 100, 1.000, ... |
| Hexadezimal | 16    | 0-9, A-F | 1, 16, 256, 4.096, ... |

# Binärsystem

- Rechner stellen Zahlen im Binärsystem dar, weil sich Ziffern 0 und 1 gut darstellen lassen:
  - 0: Strom aus/Kondensator entladen
  - 1: Strom an/Kondensator geladen
- Beispiel:
  - $0010\ 1010_2 = 42_{10}$
  - $0111\ 1110\ 0010_2 = 2018_{10}$
- Eine Ziffer = 1 Bit, kleinste Informationseinheit



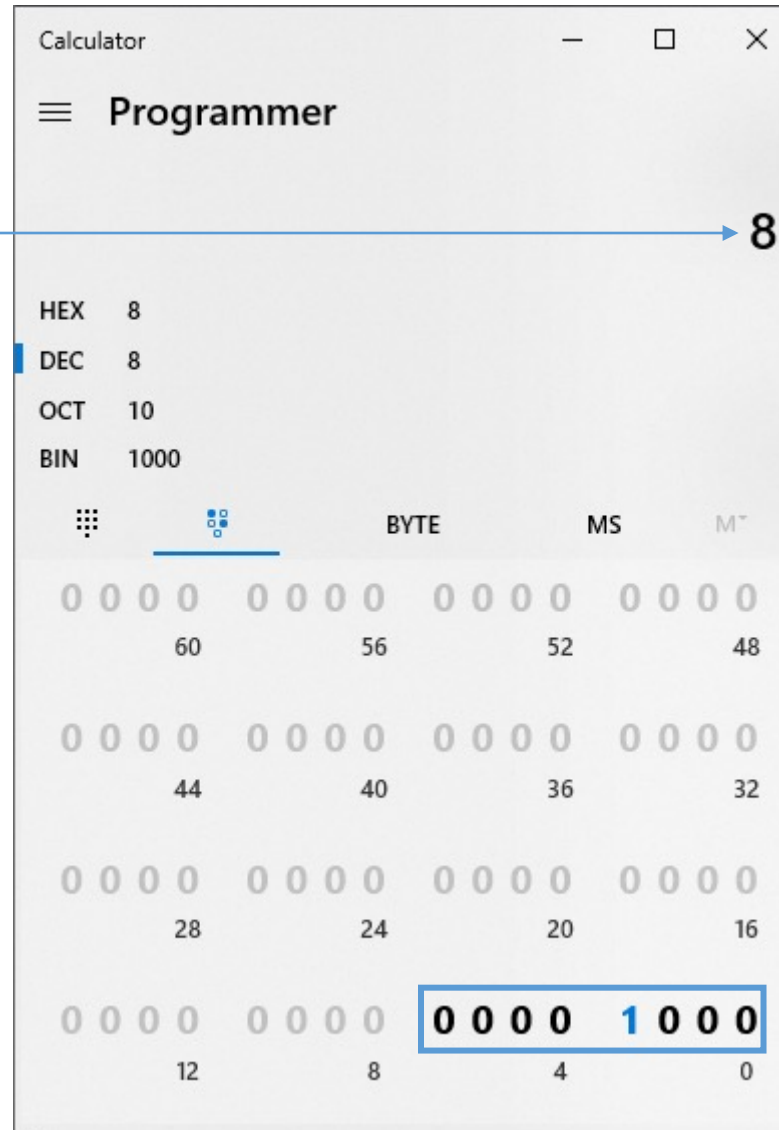
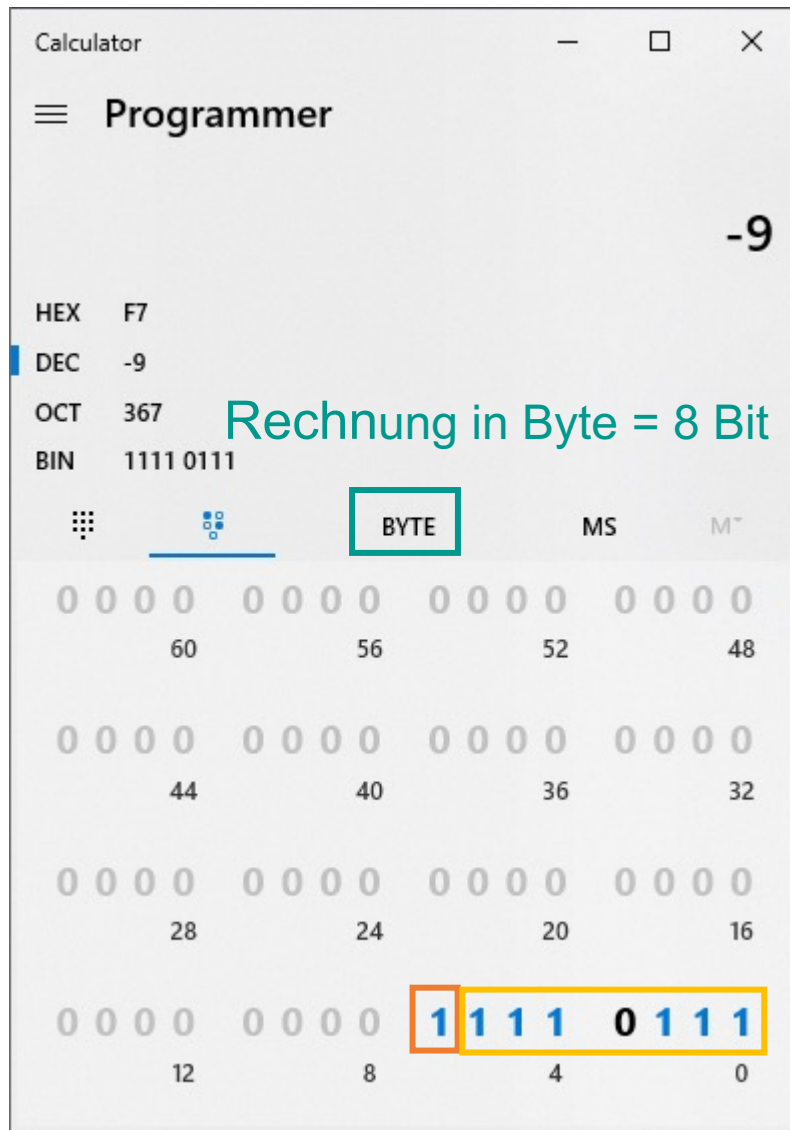
# Vorzeichendarstellung

- Ein Bit für Vorzeichen (+/-)
- Restliche Bits für Betrag
- Bei  $n$  Bits:
  - 1 Bit Vorzeichen,  $n-1$  Bits Betrag
  - Wertebereich ca.  $[-2^{n-1}; 2^{n-1}]$
- Negative Zahlen: Betragsdarstellung als sog. Zweierkomplement (mehr dazu in VL Hardware- und Systemgrundlagen HaSy)

# Ganzzahltypen

| Typ   | Länge  | Kleinste darstellbare Zahl | Größte darstellbare Zahl  |
|-------|--------|----------------------------|---------------------------|
| byte  | 8 Bit  | -128                       | 127                       |
| short | 16 Bit | -32.768                    | 32.767                    |
| int   | 32 Bit | -2.147.483.648             | 2.147.483.647             |
| long  | 64 Bit | -9.223.372.036.854.775.808 | 9.223.372.036.854.775.807 |

- erstes Bit bestimmt das Vorzeichen
  - Wenn 0 → Zahl positiv
  - Wenn 1 → Zahl negativ, Wert der Zahl ist Zweierkomplement + 1
  - erlaubt Übergang negativ → positiv ohne Sonderrechenregeln



Der Windows-Taschenrechner als  
Web-App (für Mac/Linux-User):  
<https://calculator.platform.uno/>

Vorzeichen Betrag als Zweikomplement Bits getauscht + 1 = Wert

# Overflow/Underflow

- Was passiert, wenn der Zahlenbereich verlassen wird?

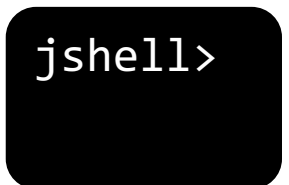
| Typ | Länge  | Kleinste darstellbare Zahl | Größte darstellbare Zahl |
|-----|--------|----------------------------|--------------------------|
| int | 32 Bit | -2.147.483.648             | 2.147.483.647            |

-1  
↓  
?

+1  
↓  
?

```
int max = 2147483647;  
int result = max + 1;
```

```
int min = -2147483648;  
int result2 = min - 1;
```

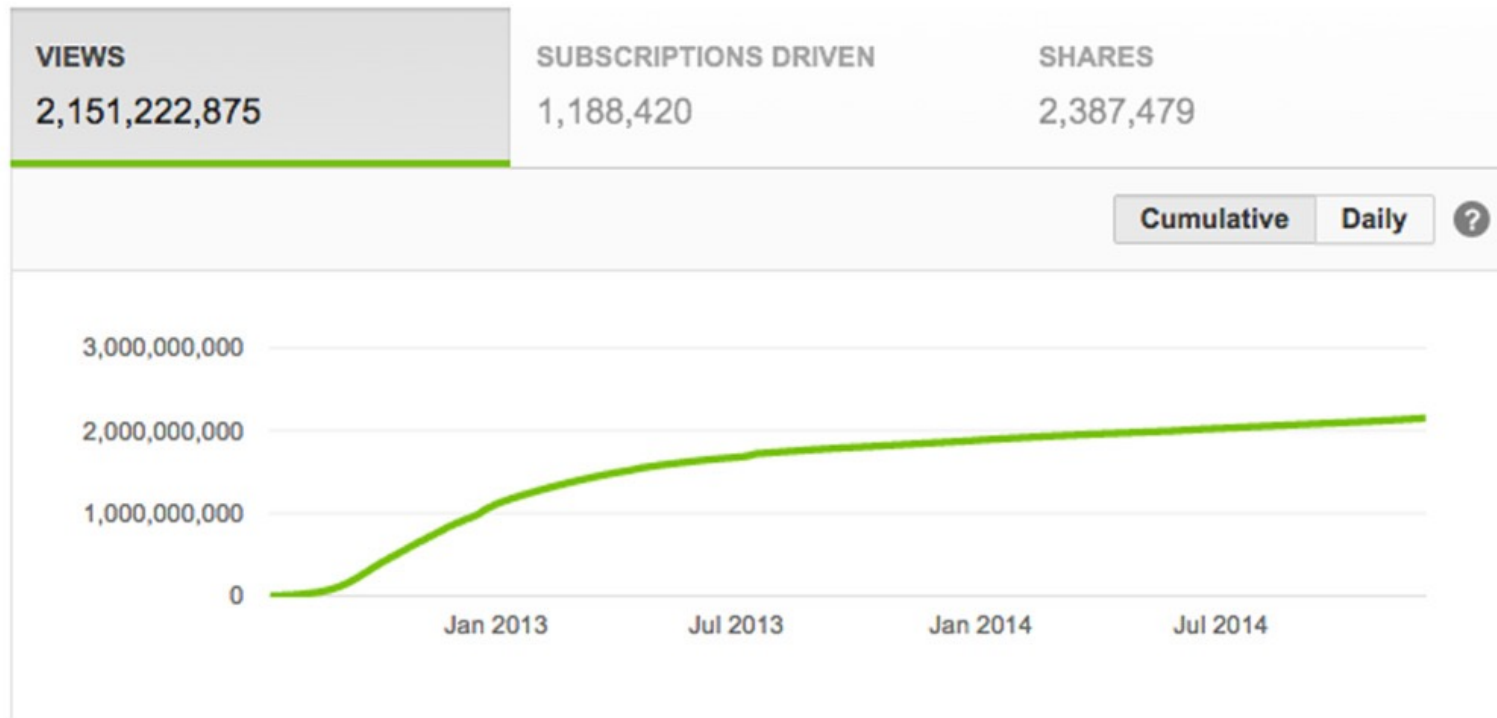


code01.  
OverUnder  
Flow

# YouTube learns about integer overflow, Gangnam Style!

We never thought a video would be watched in numbers greater than a 32-bit integer (=2,147,483,647 views), but that was before we met PSY. "Gangnam Style" has been viewed so many times we had to upgrade to a 64-bit integer (9,223,372,036,854,775,808)!

**Video statistics** Through Dec 2, 2014 ?



# Gleitkommazahlen

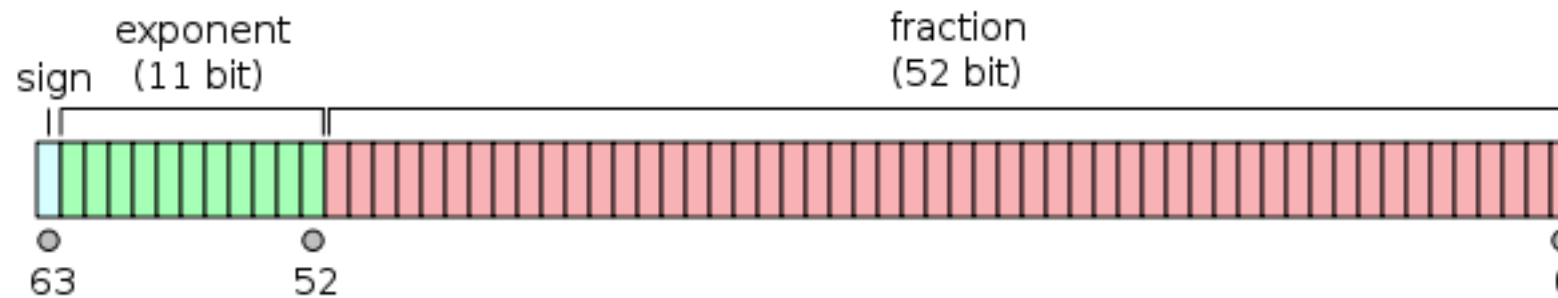
- Darstellung als Vorzeichen plus Binärzahlenpaar
  - Exponent (*exponent*) -3
  - Mantisse (*fraction*) 19
- Tatsächlicher Wert:
  - (Vorzeichen) Mantisse  $\times 2^{\text{Exponent}}$   $19 \cdot 2^{-3} = 19 \cdot 1/8 = 19/8 = 2 \frac{3}{8} = 2,375$
- Details rechnerabhängig, weit verbreitet aber Standard IEEE 754



# Gleitkommazahltypen

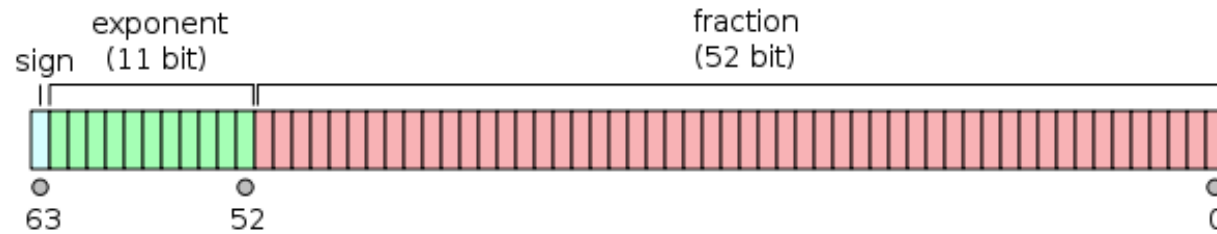
- Größerer Wertebereich als Integer, aber weniger genau

| Typ    | Länge  | Kleinste darstellbare Zahl | Größte darstellbare Zahl |
|--------|--------|----------------------------|--------------------------|
| float  | 32 Bit | ca. $(-2) \times 2^{127}$  | ca. $2 \times 2^{127}$   |
| double | 64 Bit | ca. $-10^{308}$            | ca. $10^{308}$           |



- „Nur“ ca. 8 (float) und ca. 16 (double) Dezimalstellen Genauigkeit (vor und nach dem Komma)

# Beispiel



| sign | exponent | fraction         | value = $(-1)^{\text{sign}} \cdot \text{fraction} \cdot 2^{\text{exponent}}$ |
|------|----------|------------------|------------------------------------------------------------------------------|
| 0    | -1       | 1 ( $1_2$ )      | $(-1)^0 \cdot 1 \cdot 2^{-1} = 1/2$                                          |
| 0    | 3        | 5 ( $101_2$ )    | $(-1)^0 \cdot 5 \cdot 2^3 = 5 \cdot 8 = 40$                                  |
| 1    | -3       | 19 ( $10011_2$ ) | $(-1)^1 \cdot 19 \cdot 2^{-3} = -19 \cdot 1/8 = -19/8 = -2.375$              |

- **Exakte Darstellung** nur möglich für Zahlen, die sich als Summe von Zweierpotenzen (ggf. negativer Exponent) darstellen lassen.
- $2,375 = 2 \frac{3}{8} = 1 \cdot 2^1 + 0 \cdot 2^0 + 0 \cdot 2^{-1} + 1 \cdot 2^{-2} + 1 \cdot 2^{-3}$
- Für  $1/10$  nur **Annäherung** möglich, z.B.

|   |    |                   |                                                          |
|---|----|-------------------|----------------------------------------------------------|
| 0 | -9 | 51 ( $110011_2$ ) | $(-1)^0 \cdot 51 \cdot 2^{-9} = 51/512 \approx 0.099609$ |
|---|----|-------------------|----------------------------------------------------------|

- Ähnlich Dezimal-Kommazahlen! Hier lässt sich  $1/3$  nicht als exakte Kommazahl darstellen, sondern nur über  $0,333333333$  annähern.



code01.  
DoublePrecisionIssues